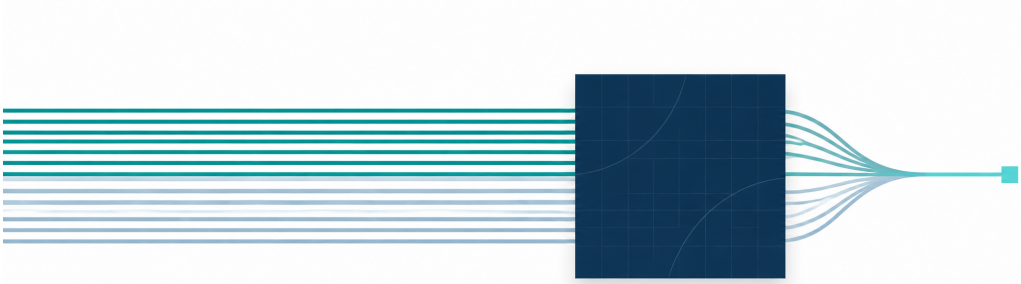
香山昆明湖 V2

自定义向量点积指令

第二阶段：指令设计与基础测试

设计、验证与性能评估



三阶段工程技术报告

2026 年 11 月 6 日修订

目录

第二阶段：指令设计与基础测试 **1**

2.1 设计目标与实施范围 **1**

2.2 指令语义与编码 **1**

2.2.1 八项有符号点积与结果布局 1

2.2.2 数值范围与中间位宽 3

2.2.3 编码与合法执行状态 3

2.3 数据通路与弹性两槽控制 **3**

2.3.1 乘积槽与结果槽 3

2.3.2 时钟、复位与连续流动 5

2.3.3 背压、重定向与事务守恒 6

2.4 RTL 实现与处理器接入 **9**

2.4.1 关键实现语句 9

2.4.2 译码、配置依赖与写回 9

2.4.3 共享 VFEX4 的派发与门控 10

2.5 基础功能与分层验证 **12**

2.5.1 正数点积测试 13

2.5.2 负数点积测试 13

2.5.3 边界值点积测试 14

2.5.4 算术数据通路测试汇总 14

2.5.5 响应配对与复查位置 15

2.6 设计取舍与扩展 **16**

2.6.1 两槽结构与结果保存 16

2.6.2 十六项 INT8 点积的功能进展 16

2.6.3 元素位宽的参数化验证 17

2.7 实现与验证摘录 **17**

2.8 源码与证据入口 **21**

第二阶段：指令设计与基础测试

2.1 设计目标与实施范围

点积把两组数据逐项相乘，再将乘积求和，是量化矩阵运算的重要计算步骤。本设计在昆明湖 V2 已有向量路径中增加八项 INT8 点积指令，将乘法与归约组合为专用数据通路，同时复用处理器的译码、寄存器重命名、调度、写回和精确提交机制。

通用构建默认采用单请求实现，弹性两槽实现作为可选优化保留。本章重点介绍两槽实现的算术结构、背压和取消处理，并通过正数、负数及边界值测试展示结果。第三阶段的主要周期和成本数据对应明确标识的八项两槽受测版本；十六项扩展的最新功能验证列在本章末尾。这样的组织覆盖赛题 2.1 的设计说明、2.2 的 RTL 实现和 2.3 的三类基础测试。

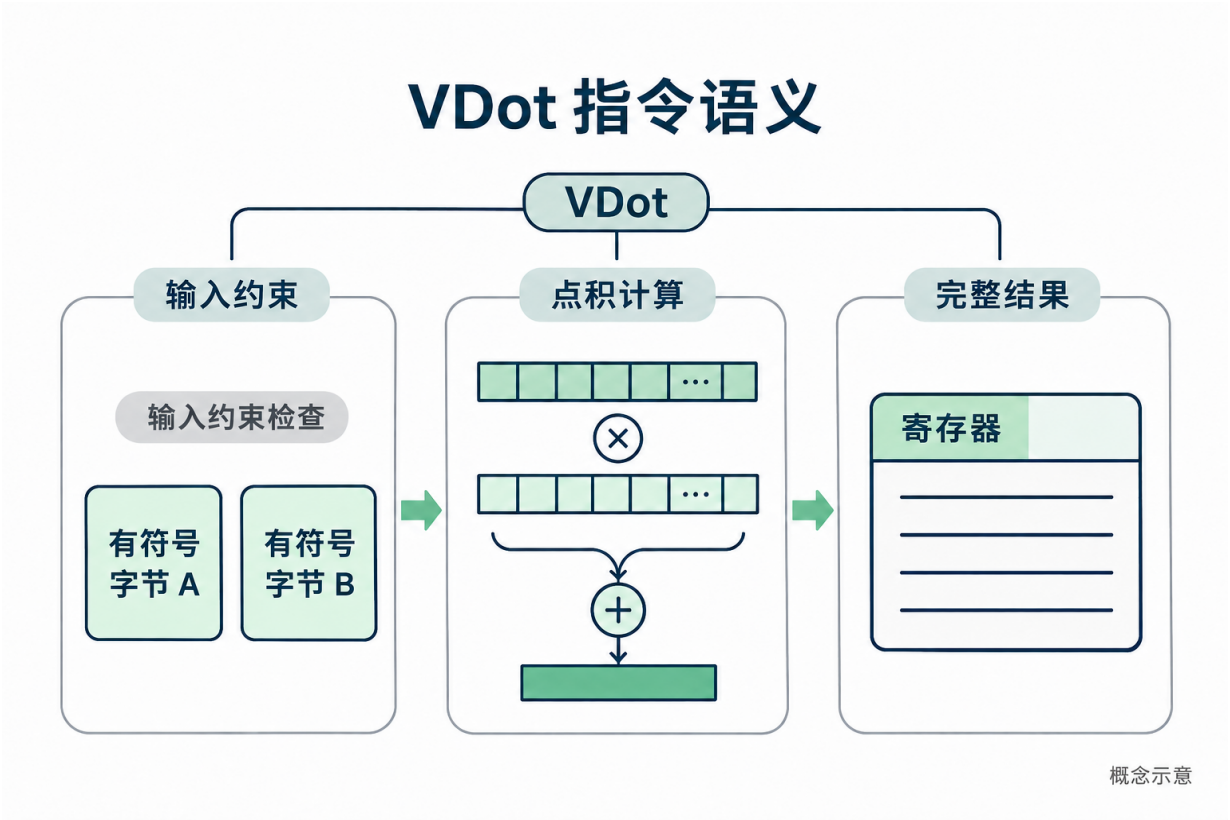
2.2 指令语义与编码

2.2.1 八项有符号点积与结果布局

本指令采用 128 位向量寄存器（VLEN=128）、8 位元素（SEW=8）和单寄存器分组（LMUL=1）；有效元素数为 8（VL=8），起始执行索引为 0（vstart=0）。指令读取两个源寄存器的低 8 个字节，将每个字节解释为有符号整数，计算

$$S = \sum_{i=0}^{7} \mathrm{int8}(vs1[8i+7:8i])\,\mathrm{int8}(vs2[8i+7:8i]).$$

目的低 32 位保存 S 的二补码，高 96 位写零。运算从零开始累加，读取指令执行前的源值，因此支持目的与源寄存器重合，也支持使用 v0 作为普通源或目的。每个源以 128 位向量传入数据通路，八项算术使用其中低 64 位的 8 个元素。



概念示意：输入约束、点积与结果写回。技术依据见图 2-1。

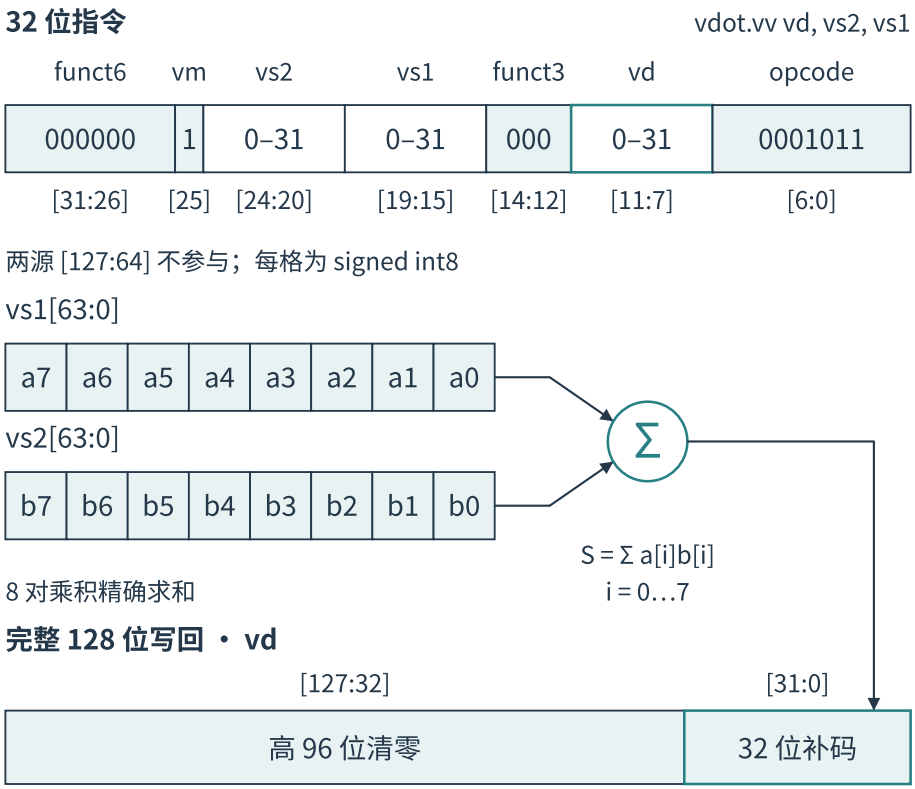


图 2-1　八项点积指令字段与完整结果布局

两个源各提供 8 个有符号字节；目的低 32 位保存点积结果，高 96 位写零。

2.2.2 数值范围与中间位宽

有符号 8 位数的范围为 -128 至 127，单个乘积范围为 $[-16256, 16384]$，八项和范围为 $[-130048, 131072]$。乘积使用 16 位；按二项、四项、八项归并时，精确位宽依次增加为 17、18、19 位。正端点 131072 比 18 位有符号数的最大值大 1，因此极值用例能直接检验归约过程中是否保留了完整进位。

归约结果先符号扩展为 32 位，再与 96 位零拼接。负数的符号由低 32 位字段表达，例如 -8 的低 32 位为 `0xfffffff8`，完整 128 位结果仍以 96 个零作为高位。内部精确和的 19 位、结果槽的 32 位和公开写回的 128 位分别承担算术、存储和架构接口三种作用。

2.2.3 编码与合法执行状态

指令采用 custom-0 编码，`MATCH=0x0200000b`、`MASK=0xfe00707f`。

字段	位段	含义或取值
funct6	31:26	`000000`
vm	25	`1`
vs2、vs1	24:20、19:15	两个向量源寄存器编号
funct3	14:12	`000`
vd	11:7	向量目的寄存器编号
opcode	6:0	custom-0，`0x0b`

汇编记法为 `vdot.vv vd, vs2, vs1`。

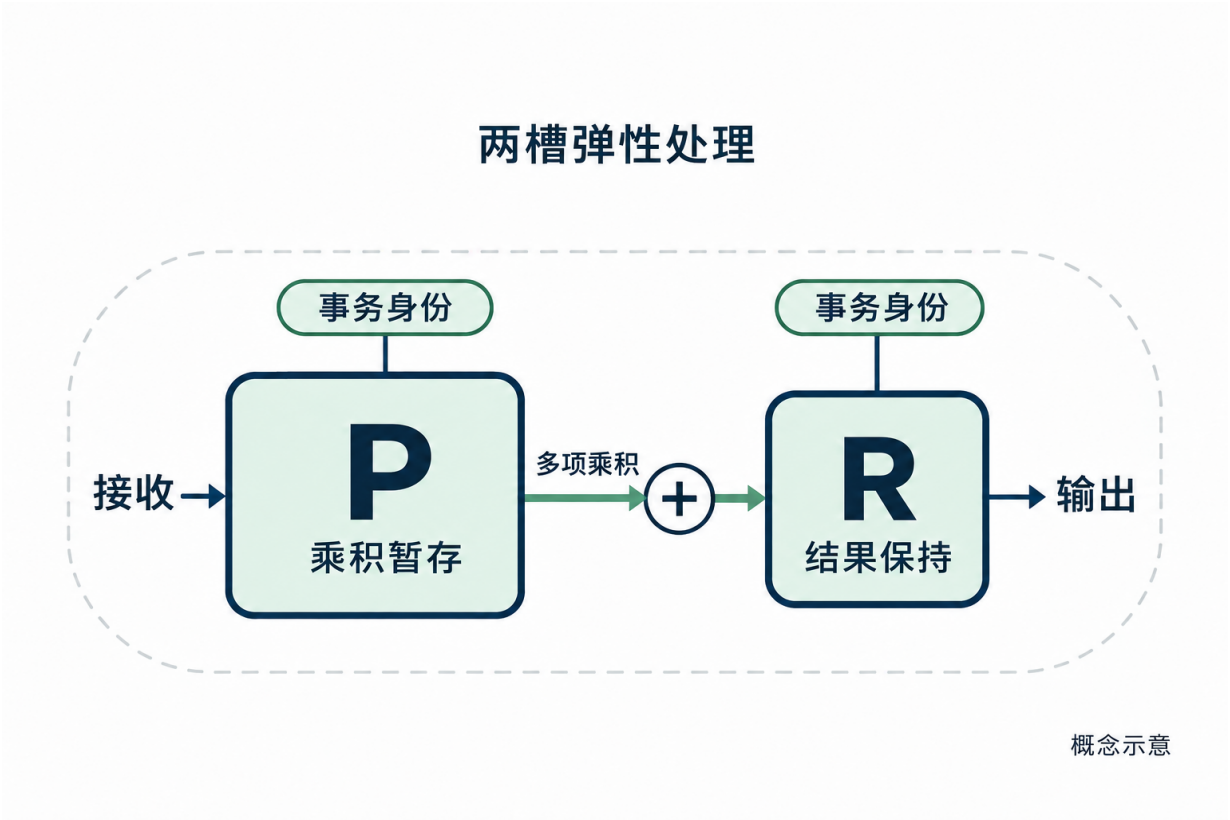
例如 `vdot.vv v10,v8,v9` 对应 `0x0284850b`。程序通过经过校验的编码宏或机器字生成该指令。

向量功能启用、vill=0、vm=1 以及本节的 SEW、LMUL、VL 和 vstart 共同确定合法状态，四种 vta/vma 策略组合均适用。合法执行完成一次完整目的写回，并保持 VL、vtype、vlenb、vxrm、vxsat 和 vcsr。非法请求携带 illegal-instruction 异常完成，同时关闭向量与 v0 目的写使能，由处理器的精确异常机制处理。

2.3 数据通路与弹性两槽控制

2.3.1 乘积槽与结果槽

点积数据通路包含 8 个有符号 8×8 乘法器、乘积槽 P、归约网络和结果槽 R。P 保存 8 个 16 位乘积及这笔请求的控制、性能信息；R 保存 32 位结果及同一请求的身份。结果从 R 输出时补入 96 位零，兼顾紧凑的内部状态和完整的向量写回接口。



概念示意：乘积暂存与结果保持。技术依据见图 2-2。

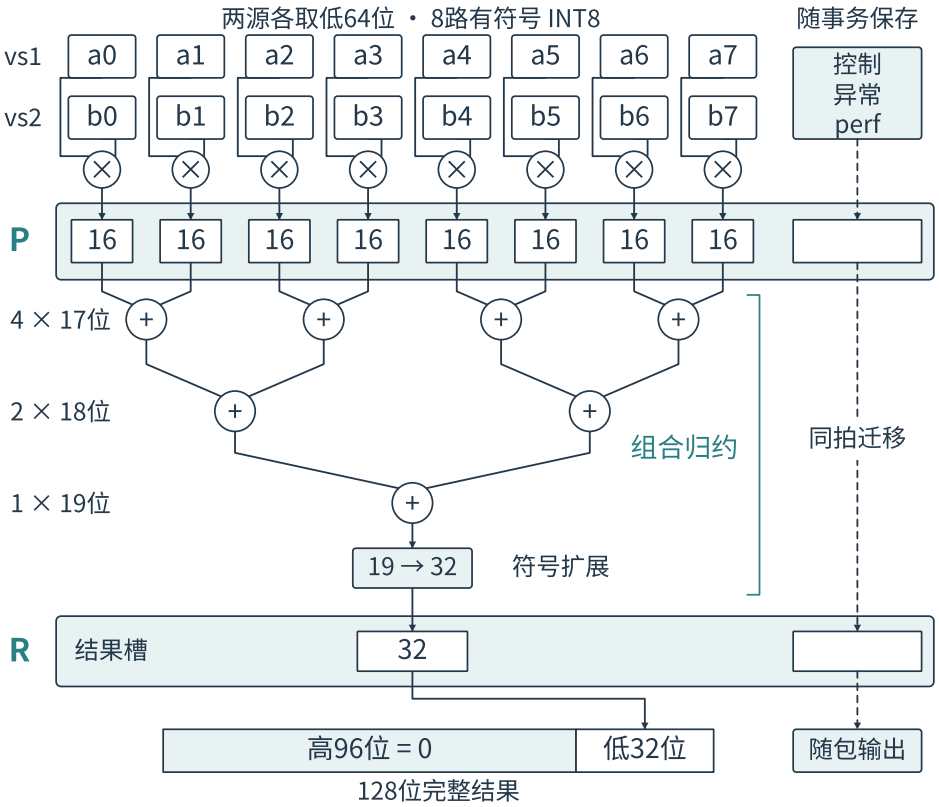


图 2-2　八项点积的乘积槽、归约网络和结果槽

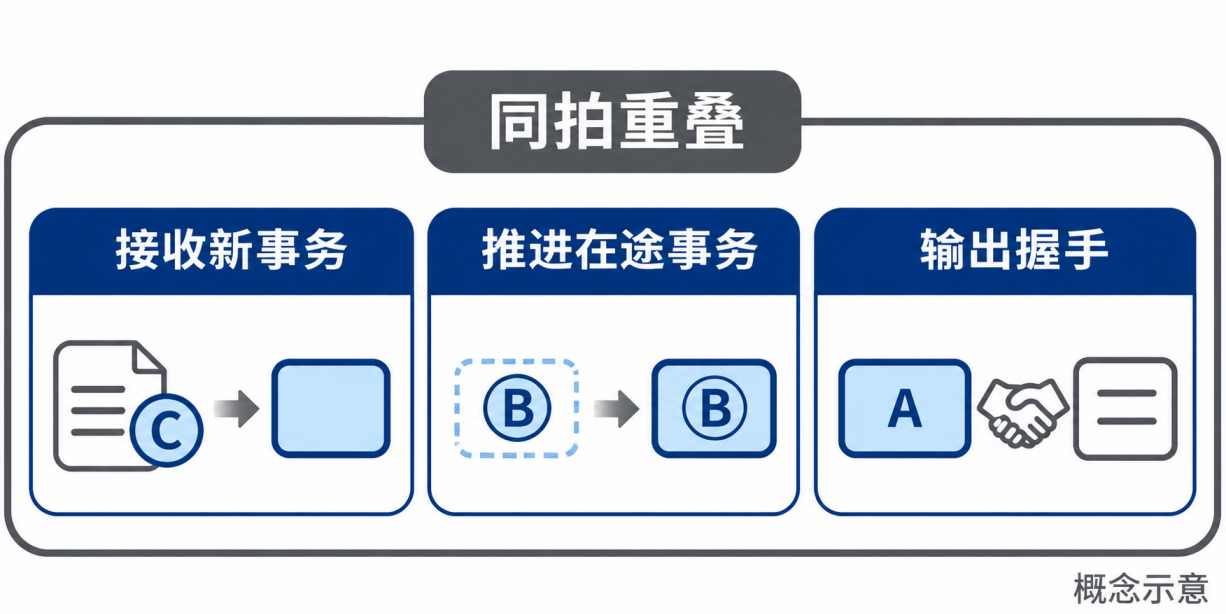
P 保存 8 路 16 位乘积，R 保存 32 位结果；控制与目的身份随数据流动，输出接口为 128 位。

两槽的有效位表示各自是否保存一笔请求。重排序缓冲区（ROB）编号、目的物理寄存器、向量或 v0 写使能、异常信息以及性能字段与数据同时移动，使写回始终能够找到正确的目的。输出停顿时，新输入的变化与 R 中已有请求相互独立。

2.3.2 时钟、复位与连续流动

VDot 功能单元（FU）接入共享执行单元 VFEX4，后文将 VFEX4 称为父级。实现沿用功能单元所在的单时钟域和父级复位连接。P、R 在时钟上升沿更新；复位清除两槽有效位，并抑制复位期间的输入和输出握手。在父级选择并允许接收 VDot、独立请求连续供给、下游持续就绪且没有复位或重定向时，请求于 E0 进入 P，E1 完成归约并进入 R，E2 是最早的输出握手沿。连续请求的发起间隔为 1 拍。

P 状态	R 状态	下游状态	本拍动作
空	空	任意	接受新请求到 P
有效	空	任意	原 P 进入 R，同时可接收下一请求
空	有效	反压	保持 R，P 可接收一笔请求
有效	有效	反压	保持两槽，暂停接收
有效	有效	就绪	原 R 输出、原 P 进入 R、新请求进入 P



概念示意：接收、推进与输出的重叠。技术依据见图 2-3。

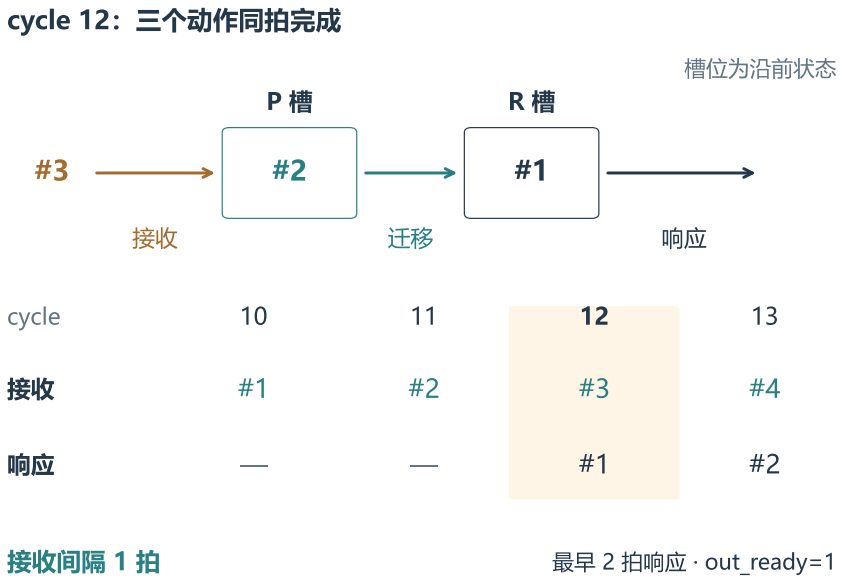


图 2-3 连续请求在两槽中的逐拍流动

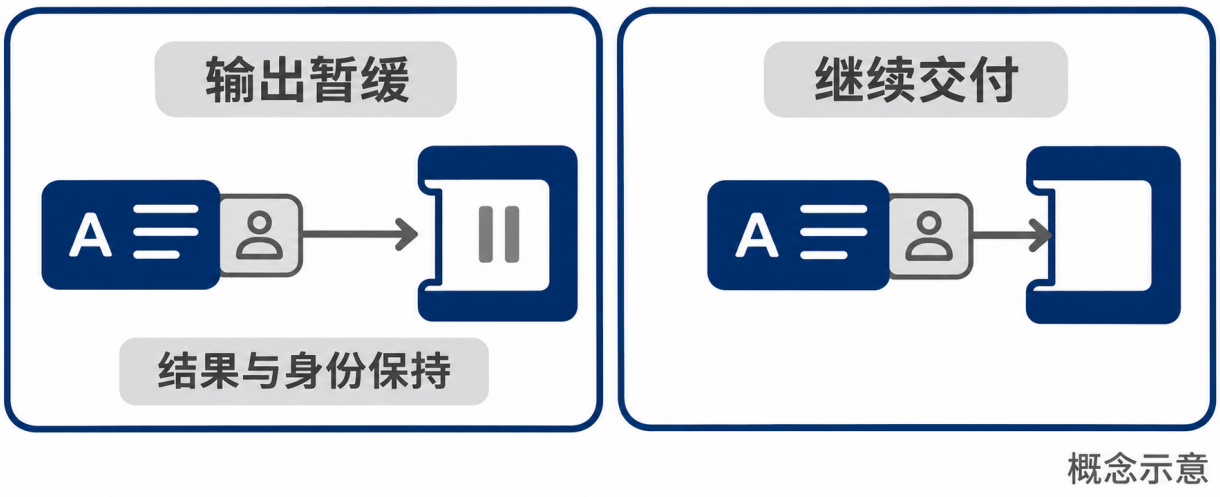
持续供给且下游就绪时，旧 R 输出、旧 P 推进和新请求接收可以在同一拍重叠；最早响应为接受后 2 拍，发起间隔为 1 拍。

子单元就绪与父级准入分别判定。槽位腾出提供接收容量，父级仍根据当前选择与共享资源占用决定何时派发新请求。

时延描述一笔请求从接受到响应的距离，发起间隔描述相邻请求的接受节奏。两槽结构支持连续独立点积；程序的最终周期还由数据依赖、调度、访存和写回共同决定。功能单元继续通过 valid/ready 报告完成，反压时可自然延后响应。

2.3.3 背压、重定向与事务守恒

R 有效而下游未就绪时，数据、目的和控制包一起保持。P 可利用空槽再接收一笔请求；若两槽均占用且 R 仍受阻，输入 ready 撤销，直到 R 输出或取消腾出空间。



概念示意：等待期间保持事务。技术依据见图 2-4。

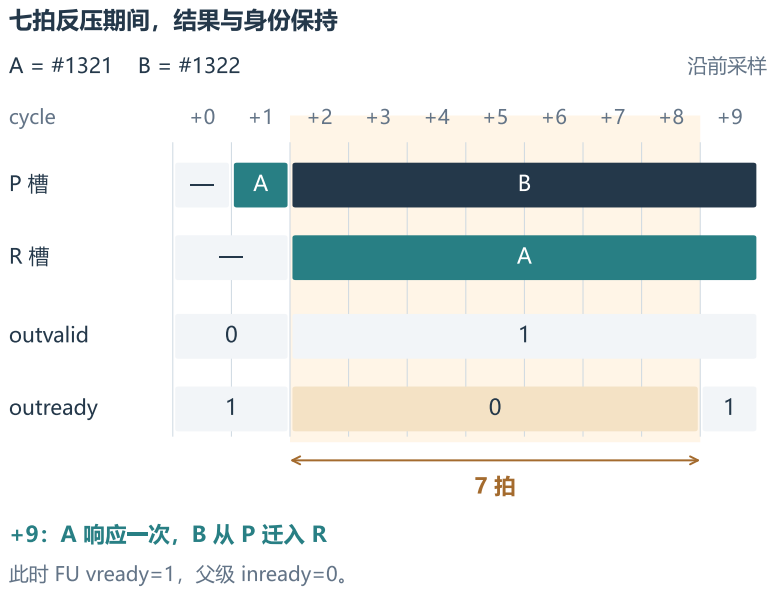


图 2-4 背压期间的数据与目的身份保持

R 在输出阻塞期间保存完整响应；空闲 P 可继续接收一笔请求，满槽后向上游施加反压。

重定向按两个槽各自的 ROB 年龄独立判断取消，保留较老的幸存请求，并在该拍暂停新请求接收。被取消的 R 停止输出，被取消的 P 释放待归约状态；复位的优先级最高。ROB 环回场景使用完整指针年龄关系确定先后。



概念示意：取消与复位的处理边界。技术依据见图 2-5。



图 2-5 部分取消、全部取消与复位恢复

分别判断 P、R 中的请求是否被重定向命中，展示幸存请求、取消清空和复位后重新接收。

每次验证在统一时钟坐标下检查

$$N_{\text{accept}} = N_{\text{response}} + N_{\text{cancel}} + N_{\text{reset}} + N_{\text{inflight}}, \qquad 0 \le N_{\text{inflight}} \le 2.$$

该关系将每笔已接受请求归入响应、取消、复位丢弃或当前占用之一，并在运行结束时检查两槽排空。每笔响应还核对完整 128 位结果、ROB 和目的身份。



概念示意：保留、取消与继续处理。技术依据见图 2-6。

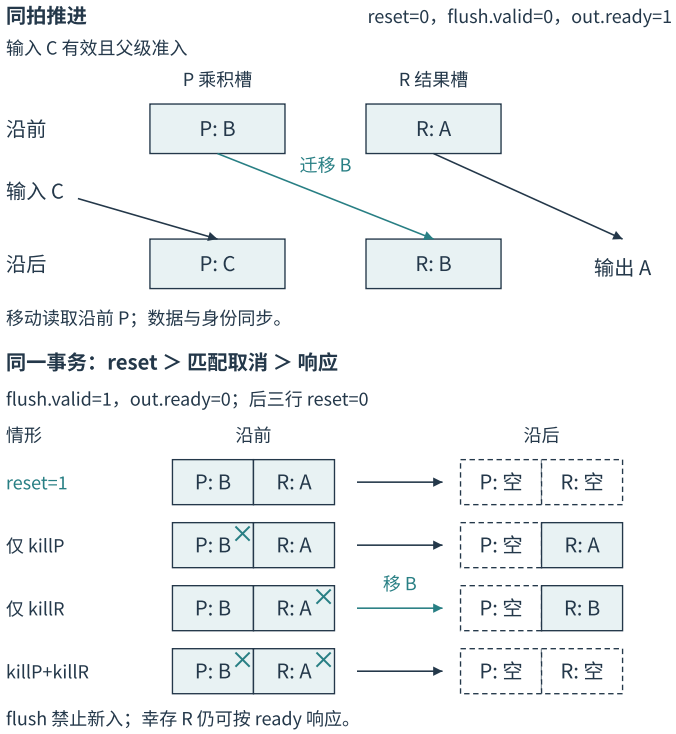


图 2-6 两槽同拍推进与取消、复位处理

满槽时原 R 输出、原 P 推进与新请求接受可在同一拍进行；取消与复位示例说明被撤销请求和幸存请求的处理。

2.4 RTL 实现与处理器接入

2.4.1 关键实现语句

算术部分位于 src/main/scala/xiangshan/backend/fu/vector/VDotDatapath.scala，乘积和每级归约按有符号类型计算，并逐级保留进位。wrapper/VDotElastic.scala 保存 P、R 的控制和数据。下面摘示从 P 转入 R 的关键语句：

```
when(move) {
  rPacket.ctrl := pCtrl
  rPacket.perfDebugInfo := pPerf
  rPacket.res.data :=
  Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
    0.U(32.W), reduce.io.result(31, 0))
}
```

这些赋值读取本拍边沿前的 P 内容，因此同拍接收的新请求会进入 P，原请求的结果和身份则进入 R。公开输出使用：

```
io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

结果寄存器的保存位宽为 32 位，高 96 位由输出组合逻辑补零。

2.4.2 译码、配置依赖与写回

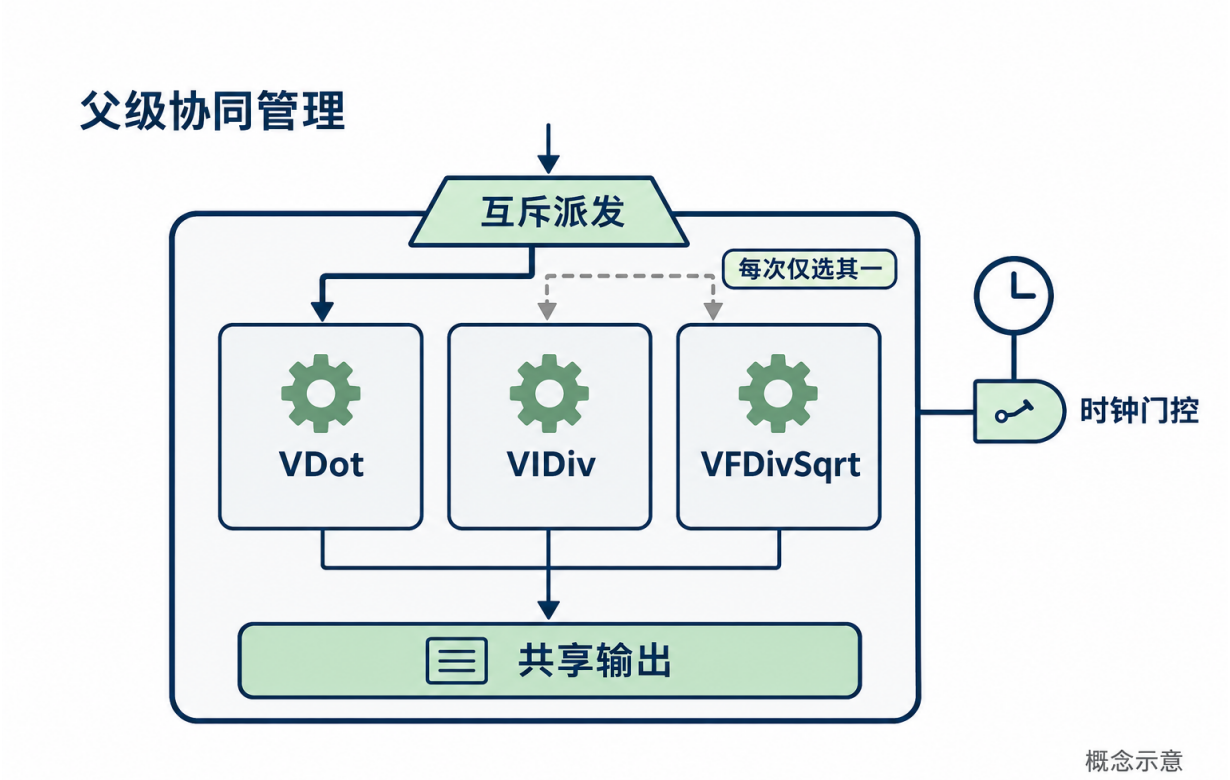
精确编码匹配后，译码器将 vdot 归入向量运算，生成单个微操作并建立两个向量源依赖。实际 VL 通过重命名后的依赖传递，SEW、LMUL 和其他向量控制随指令保存。这样，多条向量配

置与运算交错时，每条点积仍使用与自己对应的配置。

旧目的值不参与累加，译码去除相应的旧目的依赖；向量和 v0 的目的分配、异常信息及最终写回沿用现有处理器机制。选择两槽构建时显式启用 `VDOT_O2_ELASTIC=1` 和 `VDOT_T14_FUSED_INPUT=1`，并采用对应配置清单。工程同时保留通用单请求路径，便于按构建版本比较和回退。

2.4.3 共享 VFEX4 的派发与门控

VDot 与整数 DIV、浮点 DIV/SQRT 共用 VFEX4 父级。父级根据选择、实际占用和被选功能单元的 ready 进行原子派发，使父级和子单元接受的是同一笔请求。VDot 两槽排空后可以切换到 DIV；DIV 在途期间保存自己的 ROB 身份和占用状态。输出按唯一有效完成来源选择共享写回。



概念示意：VFEX4 父级派发与共享输出管理。技术依据见图 2-7。

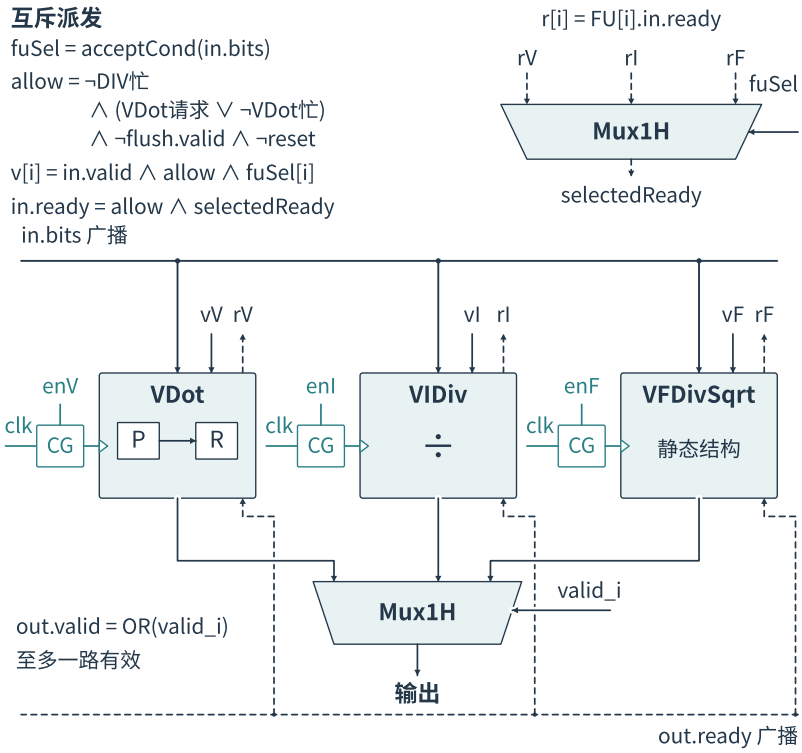
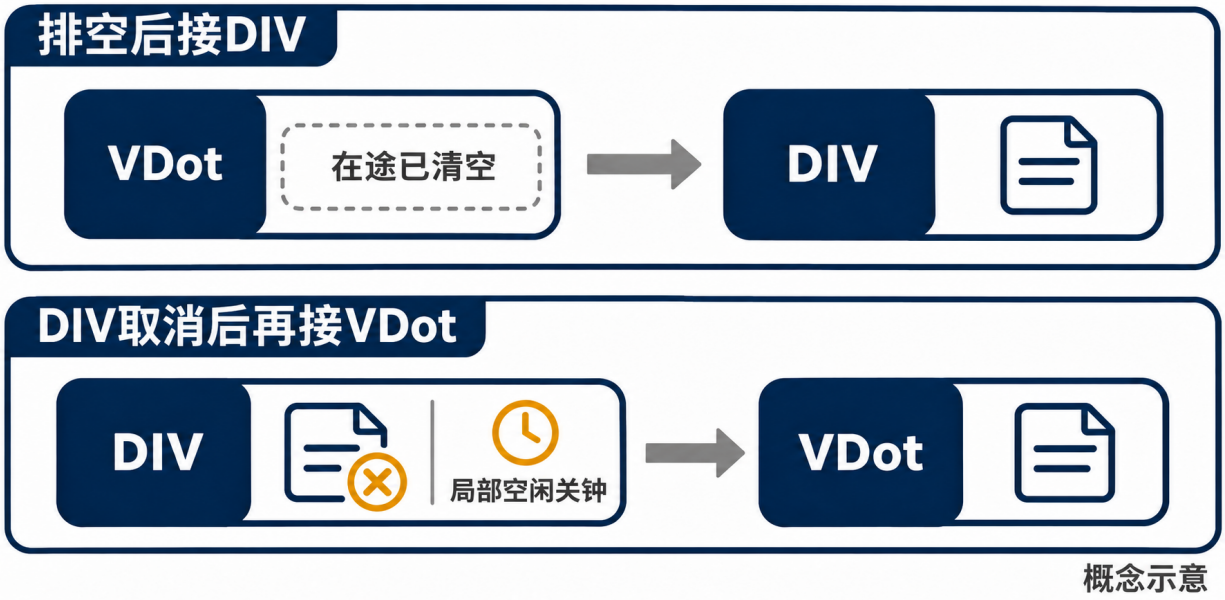


图 2-7 VFEX4 中的派发、资源互斥与共享写回

VDot、整数 DIV 和浮点 DIV/SQRT 共享父级；各自的真实占用决定派发和时钟保持。

时钟使能覆盖存活请求、当前输入接受和必要的取消、复位清理。对应验证包含满槽背压、部分取消、ROB 环回、关钟后的复位恢复，以及 VDot 和两类 DIV 之间的切换。受测的 DIV 取消修复在匹配请求撤销后释放时钟保持，使父级能够进入空闲并接受后续工作。



概念示意：功能切换与时钟管理。技术依据见图 2-8。

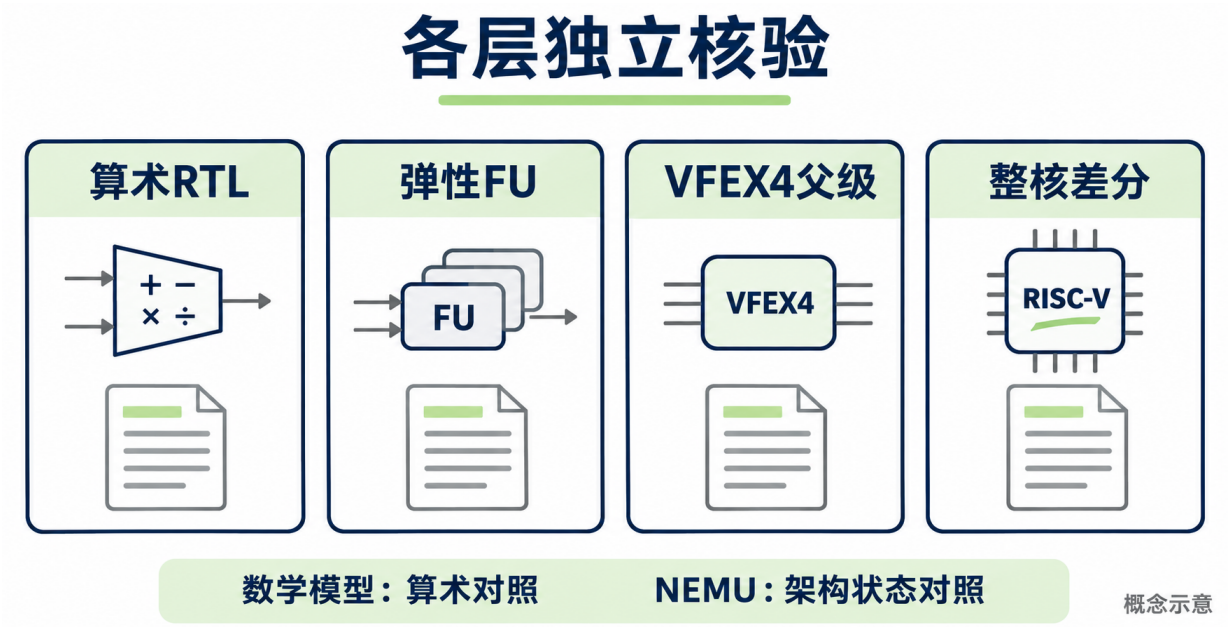


图 2-8　共享资源切换、取消与时钟保持

按存活请求保持时钟，在取消或完成后释放相应占用，随后恢复派发。

2.5　基础功能与分层验证

验证首先用独立数学计算确定完整 128 位结果，再依次检查算术 RTL、带握手的功能单元、真实父级和全核执行。算术测试关注有符号乘法、极值和逐级位宽；FU 与父级测试加入背压、取消、复位、目的身份和 DIV 交互；整核协同验证进一步检查译码、架构状态及提交结果。



概念示意：不同层级的验证视角。技术依据见图 2-9。

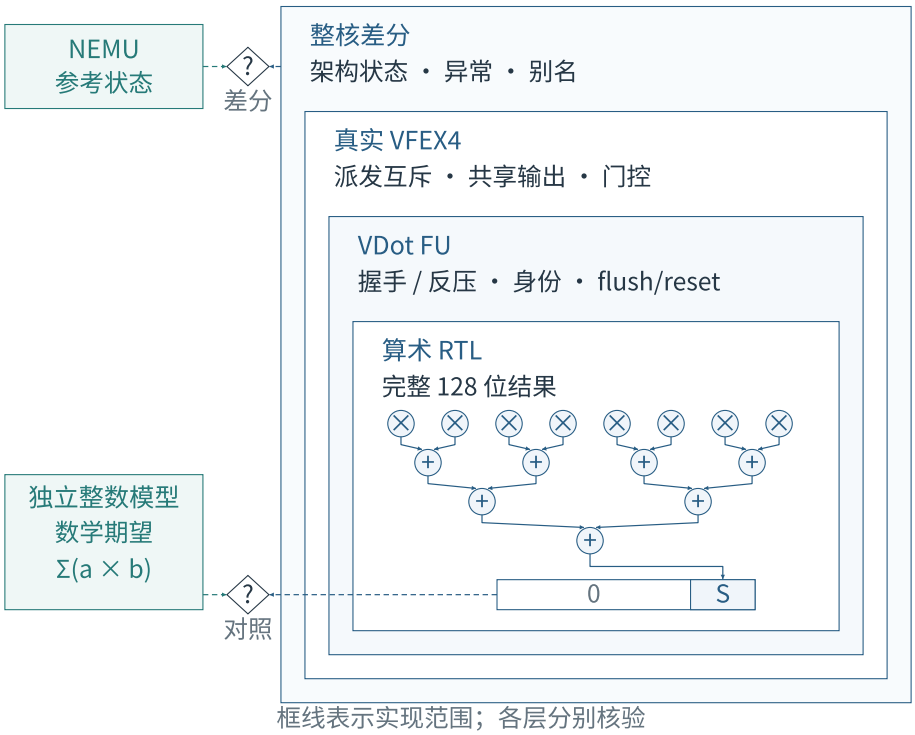


图 2-9 从数学结果到整核执行的验证层次

模型、算术 RTL、FU、真实 VFEX4 父级和整核各检查对应机制；完整 128 位结果贯穿各层。

以下三类测试对应赛题 2.3.1 至 2.3.3。图中的输入与响应取自固定八项两槽版本的全核观察记录，按 seq、ROB 和目的物理寄存器配对；第三阶段说明该版本的架构差分验证。

2.5.1 正数点积测试

两组有效输入均为 $[1, 2, 3, 4, 5, 6, 7, 8]$，八项平方和为 204。全核观察记录 `functional-2` 中，实际完整 128 位结果为 0x000000000000000000000000000000cc，与预期一致。源高 64 位填入 A5 哨兵，用于检查硬件是否准确选择了低 8 个元素。

```
[POSITIVE] functional-2  seq=1  ROB=0:39  pdest=103
vs1 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
expected sum      204
expected (128-bit) 0x000000000000000000000000000000cc
actual   (128-bit) 0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
source: clean-02/fullcore-run-01/runs/functional-2/run.log
SHA256: 90b36b19e9d141e1b0bdad6bed461b4dbde4e534798f5fbdaa8819d9e2ef3211
```

图 2-10 正数点积：预期与实际结果均为 204

两个源的有效元素为 1 至 8，响应低 32 位为 `000000cc`，高 96 位为零。

2.5.2 负数点积测试

有效输入为八个 -1 与八个 1，预期点积为 -8。`functional-0` 记录的完整结果为 0x0000000000000000000000000000fffffff8。低 32 位以二补码表达负数，高 96 位保持零，符合指令规定的结

果布局。

```
[NEGATIVE] functional-0  seq=4  ROB=0:12  pdest=143
vs1 (128-bit)      0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)      0xa5a5a5a5a5a5a5a501010101010101010
expected sum       -8
expected (128-bit)  0x000000000000000000000000fffffff8
actual   (128-bit)  0x000000000000000000000000fffffff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-11　负数点积：低 32 位保存负数补码

八个 −1 与八个 1 相乘求和；实际低 32 位为 `fffffff8`，高 96 位为零。

2.5.3　边界值点积测试

最大正结果由八个 −128 与八个 −128 产生。每项乘积为 16384，总和为 131072；`functional-0` 记录的完整结果为 0x00000000000000000000000000020000。这一输入检验了乘法和归约在正端点的精确性。

```
[BOUNDARY] functional-0  seq=5  ROB=0:43  pdest=66
vs1 (128-bit)      0xa5a5a5a5a5a5a5a580808080808080808
vs2 (128-bit)      0xa5a5a5a5a5a5a5a580808080808080808
expected sum       131072
expected (128-bit)  0x00000000000000000000000000020000
actual   (128-bit)  0x00000000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-12　正端点：八对最小 INT8 数产生 131072

实际低 32 位为 `00020000`，高 96 位为零，与八项和的正端点一致。

零输入测试采用八个 0 与八个 127，实际完整 128 位响应为零。另一数学边界由八个 −128 与八个 127 构成，预期总和为 −130048、低 32 位为 `0xfffe0400`；该值作为负端点的参考判据列入定向测试定义。

2.5.4　算术数据通路测试汇总

算术 RTL 测试包含 22 项定向输入和 10000 项随机输入，共 10022 项全部通过。定向输入针对符号、零值、极值和元素选择，随机输入进一步扩大数值组合覆盖。图中汇总描述算术数据通路；带背压和取消的时序行为在 FU 及父级测试中检查。

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

图 2-13 算术 RTL 的定向与随机测试汇总

22 项定向测试与 10000 项随机测试，共 10022 项全部通过。

2.5.5 响应配对与复查位置

四类已展示响应均按 seq、ROB 和 pdest 与接受记录配对，并比较完整 128 位结果。具体位置如下，可与原始图片中的输入和响应互相核对。

输入类型	日志	接受/响应周期	原始行号
正数	`functional-2/run.log`	339296 / 339298	7030–7034
负数	`functional-0/run.log`	420291 / 420293	7927–7931
最大正结果	`functional-0/run.log`	448417 / 448419	8400–8404
零输入	`functional-0/run.log`	311084 / 311086	5814–5818

零输入对应 seq=0、ROB=0:8、pdest=124。原始记录位于固定八项两槽受测版本的 clean-02/fullcore-run-01/runs/归档；本节展示执行单元响应，完整运行的架构差分结果见第三阶段。

(a) Historical combinational RTL suite counts (2026-10-12)

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

(b) Positive transaction: expected 204 and actual 128-bit response

```
[POSITIVE] functional-2 seq=1 ROB=0:39 pdest=103
vs1 (128-bit)      0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)      0xa5a5a5a5a5a5a5a50807060504030201
expected sum       204
expected (128-bit) 0x000000000000000000000000000000cc
actual   (128-bit) 0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
source: clean-02/fullcore-run-01/runs/functional-2/run.log
SHA256: 90b36b19e9d141e1b0bdad6bed461b4dbde4e534798f5fbdaa8819d9e2ef3211
```

(c) Negative transaction: expected -8 and actual 128-bit response

```
[NEGATIVE] functional-0 seq=4 ROB=0:12 pdest=143
vs1 (128-bit)      0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)      0xa5a5a5a5a5a5a5a50101010101010101
expected sum       -8
expected (128-bit) 0x000000000000000000000000fffffff8
actual   (128-bit) 0x000000000000000000000000fffffff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

(d) Boundary transaction: expected 131072 and actual 128-bit response

```
[BOUNDARY] functional-0 seq=5 ROB=0:43 pdest=66
vs1 (128-bit)      0xa5a5a5a5a5a5a5a58080808080808080
vs2 (128-bit)      0xa5a5a5a5a5a5a5a58080808080808080
expected sum       131072
expected (128-bit) 0x00000000000000000000000000020000
actual   (128-bit) 0x00000000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-14　正数、负数与边界测试的输入和结果

集中对照正数 204、负数 −8 和最大正结果 131072，观察低 32 位值及高 96 位零；算术统计与前述汇总为同一组数据。

在基础数值之外，测试还检查源高半区哨兵、旧目的哨兵、寄存器别名、v0 目的、非法配置和向量状态恢复。FU 与真实父级通过响应身份、写使能、取消和高位错误注入检查判定链，并在正常运行中核对完整响应与排空。

2.6　设计取舍与扩展

2.6.1　两槽结构与结果保存

两槽结构使乘法和归约能够与相邻请求重叠，面向连续独立点积提高供给能力。将 R 中的结果保存宽度收窄至 32 位，利用了高 96 位固定为零这一架构性质，减少了数据寄存器成本。每笔请求的 ROB、目的和控制信息则完整保留，以支持乱序执行、背压和精确取消。

共享父级的优化同时考虑正常完成与取消后的资源释放。方案选择综合功能、周期、标准单元面积和时序路径：一拍算术候选在成本筛选中暴露的父级代价，支持继续使用已有两槽实现作为对应性能表的受测对象。面积和各类路径的具体数据集中列在第三阶段。

2.6.2　十六项 INT8 点积的功能进展

在八项实现的基础上，团队增加了独立十六项点积候选。该候选以补丁和配套验证快照交付，采用 `MATCH=0x0e00000b`、`MASK=0xfe00707f`，在 e8/m1、VL=16 时消费两个 128 位源的

全部 16 个字节。八项指令继续使用原编码和 VL=8 语义。

候选复用两组八项乘积及归约模块。P 保存 16 个 16 位乘积，两组 19 位部分和相加得到 20 位精确和，再扩展为低 32 位结果。十六项和的范围为 $[-260096, 262144]$，R 仍保存 32 位，写回高 96 位为零。接收八项指令时，高八个乘积写零，使后级统一完成归约。两槽握手和取消机制沿用原结构。

2026 年 11 月 6 日，该候选完成了编译开关关闭和开启两种配置下的译码、FU 和真实父级验证。启用十六项扩展后，使用新 EMU 与匹配 NEMU 完成六例整核差分：2 例八项指令、4 例十六项指令全部通过。六例覆盖源高半区贡献、有符号正负端点、选定源目的别名、完整 128 位写回和恢复八项指令。相应 FU 与父级连续请求场景还核对了 2 拍首响应与 1 拍发起间隔。

该扩展的当前成果是指令与执行通路的功能验证。后续将通过完整 Q288 调用、与较快 RVV 实现的性能对照，以及同条件下的综合与时序分析，评价其应用价值。本报告的主要周期和成本表对应表中命名的八项受测版本。

2.6.3 元素位宽的参数化验证

另一组独立模块实验把输入元素扩展为 INT16，计算 8 项点积，并使用低 64 位结果字段、高 64 位清零的输出布局。该配置的最大正和为 2^{33}，精确累加至少需要 35 位。参数化乘法与归约模块完成 94 项定向输入和 10000 项随机输入的完整 128 位结果检查，并验证单请求包装中的反压、取消与复位行为。这组结果展示了从元素位宽到中间精度、结果字段和测试向量的一致扩展方法。

2.7 实现与验证摘录

下列摘录把前文的数据通路、状态控制和验证方法对应到具体实现与运行记录。各图分别展示源码、主机模型和逐拍日志，便于结合正文复查。

(a) Fixed 0f VDotProducts source excerpt

```
class VDotProducts extends RawModule {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val products = Output(Vec(8, SInt(16.W)))
  })

  for (lane <- 0 until 8) {
    val a = io.vs1(8 * lane + 7, 8 * lane).asSInt
    val b = io.vs2(8 * lane + 7, 8 * lane).asSInt
    io.products(lane) := a * b
  }
}

/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
```

(b) Fixed 0f VDotReduce source excerpt

```
/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
  val io = IO(new Bundle {
    val products = Input(Vec(8, SInt(16.W)))
    val result = Output(UInt(128.W))
  })

  private val sums17 = Wire(Vec(4, SInt(17.W)))
  private val sums18 = Wire(Vec(2, SInt(18.W)))
  private val sum19 = Wire(SInt(19.W))

  for (pair <- 0 until 4) {
    sums17(pair) := io.products(2 * pair) +& io.products(2 * pair + 1)
  }
  for (pair <- 0 until 2) {
    sums18(pair) := sums17(2 * pair) +& sums17(2 * pair + 1)
  }
  sum19 := sums18(0) +& sums18(1)

  private val result32 = Wire(SInt(32.W))
  result32 := sum19
  io.result := Cat(0.U(96.W), result32.asUInt)
}

/** Combinational arithmetic top for independent verification.
  * The future functional-unit wrapper instantiates the same two submodules
  * separately, with registers between them; this top has no cycle protocol.
  */
```

图 2-15　八项点积算术实现摘录

八路有符号乘法和逐级归约，对应本章的中间位宽分析。

(a) Slot registers and 32-bit result state

```
    require(!cfg.writeFflags && !cfg.writeVxsat && !cfg.flushPipe && !cfg.replayInst)

    val pipelineStatus = IO(Output(new VDotElasticStatus))

>   private val pValid = RegInit(false.B)
    private val rValid = RegInit(false.B)
    private val pProducts = Reg(Vec(8, SInt(16.W)))
    private val pCtrl = Reg(chiselTypeOf(io.out.bits.ctrl))
    private val pPerf = Reg(chiselTypeOf(io.out.bits.perfDebugInfo))
    // H1 stores only the architectural low word; the public result stays 128 bits.
    // Keep each slot's complete control/perf identity on the same transfer edge.
    private val rPacket = Reg(new Bundle {
      val ctrl = chiselTypeOf(io.out.bits.ctrl)
      val res = new Bundle {
        val data = UInt(32.W)
      }
      val perfDebugInfo = chiselTypeOf(io.out.bits.perfDebugInfo)
    })

    pipelineStatus.pValid := pValid
    pipelineStatus.rValid := rValid
    pipelineStatus.occupancy := pValid.asUInt +& rValid.asUInt

    private val illegalInput = vill || vsew =/= VSew.e8 || vlmul =/= VLmul.m1 ||
      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
```

(b) Handshake and cancellation conditions

```
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -Pattern 'private val rReady
=' -Context 5,12

      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
    private val killP = pValid && pCtrl.robIdx.needFlush(io.flush)
    private val killR = rValid && rPacket.ctrl.robIdx.needFlush(io.flush)
    private val liveP = pValid && !killP && !reset.asBool
    private val liveR = rValid && !killR && !reset.asBool
>   private val rReady = !liveR || io.out.ready
    private val move = liveP && rReady
    private val pReady = !liveP || rReady

    // Parent admission is enforced by its atomic child dispatch. The FU also
    // suppresses every new accept on redirect/reset, including unrelated flush.
    io.in.ready := pReady && !io.flush.valid && !reset.asBool
    io.out.valid := liveR
    io.out.bits.ctrl := rPacket.ctrl
    io.out.bits.perfDebugInfo := rPacket.perfDebugInfo
    io.out.bits.res := 0.U.asTypeOf(io.out.bits.res)
    io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

(c) State update and old-P transfer

```
PS C:\WINDOWS\system32>
                       > wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -SimpleMatch 'pValid := (live
P && !move) || io.in.fire' -Context 2,16

    acceptedCtrl.exceptionVec.get(ExceptionNO.illegalInstr) := illegalInput

>   pValid := (liveP && !move) || io.in.fire
    rValid := (liveR && !io.out.fire) || move

    when(io.in.fire) {
      pProducts := products.io.products
      pCtrl := acceptedCtrl
      pPerf := io.in.bits.perfDebugInfo
    }
    when(move) {
      // Reads pre-edge P even when a new request replaces it at this same edge.
      rPacket.ctrl := pCtrl
      rPacket.perfDebugInfo := pPerf
      rPacket.res.data := Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
        0.U(32.W), reduce.io.result(31, 0))
    }

    // Read-only handles for the separate G1/G2 observation harness. These carry
```

图 2-16　两槽状态与控制实现摘录

结果槽保存 32 位；有效位、握手、取消条件和状态更新共同管理请求。

Host model checks only; no RTL/NEMU execution in this command



图 2-17 独立主机模型的测试输出

35 个主机模型测试方法通过。该统计以测试方法为单位；算术 RTL 的 10022 组输入另见基础测试汇总。

Historical 0f observer log excerpt; NOT a GTKWave waveform

```
S2-05 SUPPORTING EVIDENCE: VDOT TWO-SLOT EVENT LOG EXCERPT
Historical 0f clean-02, dense-batch128, 2026-10-27 (UTC).
Observer event log excerpt; this is NOT a waveform screenshot.
Selected fields copied from the source lines listed below; fields omitted.
Source: /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/dense-batch128/run.log
Source SHA256: 18f472d75feef0378bec71c64793ffd2297bc37c47a5b501597f222da174b98f

line 146 cycle=10984 O2_STATE
  pvalid=0 pseq=0 rvalid=0 rseq=0 occupancy=0 invalid=1 inready=1 outvalid=0 outready=1 move=0
line 148 cycle=10984 O2_ACCEPT
  seq=0 rob=1:31 pdest=110 vl=8 illegal=0 vecwen=1
line 149 cycle=10985 O2_STATE
  pvalid=1 pseq=0 rvalid=0 rseq=0 occupancy=1 invalid=1 inready=1 outvalid=0 outready=1 move=1
line 151 cycle=10985 O2_ACCEPT
  seq=1 rob=1:32 pdest=111 vl=8 illegal=0 vecwen=1
line 152 cycle=10985 O2_MOVE
  seq=0 rob=1:31 pdest=110 illegal=0 vecwen=1
line 153 cycle=10986 O2_STATE
  pvalid=1 pseq=1 rvalid=1 rseq=0 occupancy=2 invalid=1 inready=1 outvalid=1 outready=1 move=1
line 155 cycle=10986 O2_ACCEPT
  seq=2 rob=1:33 pdest=112 vl=8 illegal=0 vecwen=1
line 156 cycle=10986 O2_MOVE
  seq=1 rob=1:32 pdest=111 illegal=0 vecwen=1
line 157 cycle=10986 O2_RESPONSE
  seq=0 rob=1:31 pdest=110 data=0x00000000000000000000000000020000 vecwen=1

At cycle 10986: ACCEPT seq=2, MOVE seq=1, RESPONSE seq=0.
Source result.json reports O2_DENSE_ELASTIC_AND_WINDOWS_PASS and exit_code=0.
Original batch UTC capacity audit FAIL retained; see R5/G5 acceptance RESULT.md.
```

图 2-18 连续事务的同拍重叠记录

10986 周期同时记录 ACCEPT seq2、MOVE seq1 与 RESPONSE seq0，分别对应新输入、原 P 推进和原 R 响应。

2.8 源码与证据入口

内容	阅读入口
指令语义与编码	指令规格与定向向量
算术及两槽实现	八项弹性 FU 源码及乘积与归约源码
构建与版本选择	工程版本入口与可选配置
周期、面积与 STA	八项两槽受测版本的数据与原始报告
十六项扩展	候选结构、补丁与分层验证、六例整核差分结果
元素位宽参数化	八项 INT16 独立模块及完整结果验证

工程索引保留完整源码版本、运行输入、原始日志和失败记录。正文以设计机制和结果解释为主，读者可按需要沿上述入口复查实现细节与测量依据。